BRAINWARE UNIVERSITY

ML PROJECT-BASED LEARNING REPORT (WEEK 3)

1. Basic Information

Title of the Project:	**Console-Based Management Systems (Programs 1- 3)**
Programs Covered:	**Program 1: Bank Management System** **Program 2: Hotel Booking System** **Program 3: Library Management System**
Name(s) of Student(s):	**Soumyadwip Das (BWU/BTS/25/169)** **Ankita Paul (BWU/BTS/25/180)** **Sonia Naskar (BWU/BTS/25/195)** **Joyeta Mondal (BWU/BTS/25/214)** **Sayantan Bharati (BWU/BTS/25/503)**
Programme & Semester:	**B.Tech CSE & 3rd Semester**
Course Name & Code:	**Python Programming Lab (BES09013)**
Name of the Guide/Supervisor:	**MR. PRANAB GHARAI & MR. INDRANIL DAS**

2. Introduction and Background

Week 3 of the Machine Learning project-based lab continued the series of terminal-based, menu-driven Python applications by extending the dictionary-based record-keeping approach established in earlier weeks to three independent domains: banking, hotel reservations, and library circulation. All three programs share a common design language — bordered console output via reusable box-drawing helper functions, in-memory dictionary/list-based storage, and validated, menu-driven control flow - while each models the state and business rules specific to its own domain.

This report documents all three Week 3 programs together. Each program is presented as a fully self-contained section below (introduction, objectives, problem statement, methodology, expected outcomes, core logic and output screenshot placeholder) so that the programs remain clearly distinguishable and are not mixed with one another, followed by a single combined project timeline, tools list, and reference list for the week.

3. Program 1: Bank Management System

3.1 Introduction and Background

A bank management system automates core banking operations such as account creation, balance inquiry, deposits and withdrawals, replacing manual record-keeping with a structured, menu-driven program. This program develops a terminal-based bank management application in Python that demonstrates these operations using in-memory data structures, focusing on unique account-number generation, PIN-based authentication, and safely processing deposit and withdrawal transactions inside a readable, boxed console interface.

3.2 Objectives

- Design and implement a menu-driven console application for bank account management
- Implement unique account number generation using the random and math modules
- Implement PIN-based authentication to secure account operations
- Implement deposit and withdrawal operations with balance validation
- Implement a balance inquiry feature for authenticated users
- Explore Python dictionaries and lists for in-memory data storage
- Design a consistent, bordered text-based user interface for terminal output

3.3 Problem Statement / Driving Question

How can fundamental banking operations — account creation, authentication, deposit and withdrawal — be implemented reliably in a menu-driven console application using core Python data structures and control flow, without relying on an external database?

3.4 Methodology

Program Design

- Defined a WIDTH constant and box-drawing helper functions (box_top, box_bottom, box_divider, box_line, print_boxed) for consistent bordered terminal output
- Designed an ASCII-art banner using a raw string for program branding

Data Structures

- Used a USERS dictionary keyed by account number, storing each user's name, PIN and balance
- Used an ACCOUNTS list to track issued account numbers and prevent duplicate generation

Core Banking Functions

- generate_account(): generated a unique 10-digit account number prefixed with "A", checked against existing accounts for uniqueness
- create_account(name, pin): registered a new user with a zero starting balance
- authenticate(account, pin): validated an account number and PIN combination before allowing further actions

- deposit(account, amount): increased the stored balance for an authenticated account
- withdraw(account, amount): decreased the balance only when sufficient funds were available, returning a success/failure flag

Menu-Driven Program Flow

- Implemented main() with a continuous loop presenting five options: Create Account, Check Balance, Withdraw, Deposit and Exit
- Added input validation using try/except around numeric conversions to prevent crashes from invalid amounts
- Added error handling for unregistered accounts and incorrect PIN/account combinations

3.5 Expected Outcomes / Deliverables

- A functional command-line bank management system supporting account creation, deposit, withdrawal and balance inquiry
- Randomly generated, unique account numbers for every new customer
- PIN-based authentication preventing unauthorized access to account operations
- Input validation that prevents the program from crashing on invalid numeric entries
- A consistent, readable, bordered text interface for all program output

3.6 Core Logic

The snippet below highlights two of the most important routines in the program: unique account-number generation and the balance-checked withdrawal logic.

```
def generate_account():
    while True:
        num = math.floor(random.random() * (10 ** 10))
        account_no = "A" + str(num)
        if account_no not in ACCOUNTS:
            return account_no  # ensures every account number is unique

def withdraw(account, amount):
    if amount <= USERS[account]['balance']:
        USERS[account]['balance'] -= amount
        return True   # withdrawal allowed, balance updated
    return False       # insufficient funds, transaction rejected
```

generate_account() repeatedly produces a random 10-digit number and prefixes it with "A" until a value not already present in ACCOUNTS is found, guaranteeing every account number is unique. withdraw() only reduces the stored balance when the requested amount does not exceed the current balance, returning True on success and False when funds are insufficient — this check is what prevents an account from going negative.

3.7 Output Screenshot

4. Program 2: Hotel Booking System

4.1 Introduction and Background

A hotel booking system automates room reservation management — tracking room inventory, processing bookings, and handling check-outs — replacing manual registers with a structured, menu-driven program. Continuing the terminal-application series begun with the Bank Management System, this program models multiple room categories with distinct pricing and inventory, calculates live room availability, generates unique booking records, and safely processes check-outs, all presented inside a consistent boxed console interface.

4.2 Objectives

- Design and implement a menu-driven console application for hotel room booking and management
- Model multiple room categories (Single, Double, Suite) with distinct pricing and inventory counts
- Implement real-time room availability calculation based on active bookings
- Implement room booking logic that generates a unique booking ID and computes the total cost from nights stayed
- Implement a check-out feature that releases a booked room back into availability
- Implement a feature to view all bookings along with their current status
- Explore Python dictionaries for structured, relational-style in-memory data modelling

4.3 Problem Statement / Driving Question

How can a hotel reliably track room inventory across multiple room categories, process bookings and check-outs, and compute accurate charges — all within a lightweight console application that does not depend on an external database?

4.4 Methodology

Program Design

- Reused the box-drawing helper functions (box_top, box_bottom, box_divider, box_line, print_boxed) established in the Bank Management System for consistent bordered terminal output
- Designed a new ASCII-art banner for hotel-booking branding

Data Structures

- Used a ROOM_TYPES dictionary describing each room category's price per night and total room count (Single, Double, Suite)
- Used a BOOKINGS dictionary keyed by booking ID, storing guest name, room type, nights, total cost and status

Core Booking Functions

- available_rooms(room_type): counted currently "Booked" reservations of a given type and subtracted them from total inventory to compute live vacancy
- book_room(name, room_type, nights): checked availability, generated a sequential booking ID, computed the total price as rate × nights, and stored the new booking

- checkout(booking_id): validated that a booking existed and was still "Booked", then marked it "Checked-out" to free the room

Menu-Driven Program Flow

- Implemented main() with a continuous loop presenting five options: View Rooms & Availability, Book a Room, Check-Out / Free a Room, View All Bookings and Exit
- Added input validation using try/except for the number of nights and booking ID, including a positive-value check on nights
- Added validation of the entered room type against ROOM_TYPES and graceful error messages for unavailable rooms or missing bookings

4.5 Expected Outcomes / Deliverables

- A functional command-line hotel booking system supporting room browsing, booking, check-out and booking history
- Accurate, real-time room availability calculated from active bookings
- Sequentially generated, unique booking IDs for every reservation
- Correct total-cost calculation based on room rate and nights stayed
- Input validation that prevents the program from crashing on invalid room types, nights or booking IDs
- A consistent, bordered console interface matching the style established in the Bank Management System

4.6 Core Logic

The snippet below highlights two of the most important routines in the program: live availability calculation and the room-booking logic.

```
def available_rooms(room_type):
    booked = sum(
        1 for b in BOOKINGS.values()
        if b["room_type"] == room_type
        and b["status"] == "Booked"
    )
    # vacancy = total rooms - active bookings
    return ROOM_TYPES[room_type]["count"] - booked

def book_room(name, room_type, nights):
    if available_rooms(room_type) <= 0:
        return None, f"No {room_type} rooms available."
    booking_id = 1001 + len(BOOKINGS)      # unique, sequential
    total = ROOM_TYPES[room_type]["price"] * nights
    BOOKINGS[booking_id] = {
        "name": name, "room_type": room_type,
        "nights": nights, "total": total, "status": "Booked"
    }
    return booking_id, total
```

available_rooms() counts how many rooms of a given type currently have an active "Booked" status and subtracts that from the category's total inventory to report live vacancy. book_room() first checks this

availability, and only if a room is free does it generate a new sequential booking ID, compute the total charge as price × nights, and store the reservation — this ordering is what prevents overbooking a room type.

4.7 Output Screenshot

5. Program 3: Library Management System

5.1 Introduction and Background

A library management system helps track a collection of books along with their availability, borrowers, and circulation history. This program implements such a system as a terminal-based, menu-driven Python application supporting the core library workflow: adding new books, viewing the full catalogue, issuing and returning books, and searching records by ID, title, or author. Continuing the progression from the Bank Management System and Hotel Booking System, this program applies the same dictionary-based record-keeping approach to a library context, with added emphasis on state transitions — a book moving between 'Available' and 'Issued' status.

5.2 Objectives

- Implement a menu-driven terminal application for managing a library's book records
- Develop functionality to add new books with a unique book ID, title, and author
- Implement issue and return workflows that update each book's availability status
- Provide a search feature that matches a term against book ID, title, or author
- Display a formatted, tabular view of all books and their current status
- Validate operations so books cannot be issued twice or returned when not issued

5.3 Problem Statement / Driving Question

How can a terminal-based application maintain accurate, real-time records of a library's books — tracking additions, issues, returns, and searches — while preventing invalid state changes such as issuing an already-issued book?

5.4 Methodology

Program Design

- Designed a menu-driven interface offering six options: Add Book, View All Books, Issue Book, Return Book, Search Book, and Exit
- Reused the box-drawing helper functions (box_top, box_bottom, box_divider, box_line, print_boxed) and an ASCII-art banner to present a bordered terminal UI

Book Record Management

- Developed add_book() to store each book's title, author, status, and current borrower in a dictionary (BOOKS), keyed by a unique book ID
- Implemented a duplicate-ID check when adding a book, rejecting the entry if the ID already exists
- Implemented a formatted 'View All Books' listing showing ID, title, author, status, and issued-to member in tabular form

Issue, Return & Search Logic

- Developed issue_book() to mark a book as Issued and record the borrowing member, only when the book exists and is currently Available

- Developed return_book() to mark a book back as Available and clear the borrower, only when the book is currently Issued
- Developed search_book() to perform a case-insensitive match of a search term against book ID, title, and author
- Tested all menu options, including error paths such as searching with no matches, issuing an already-issued book, and returning a book that was never issued

5.5 Expected Outcomes / Deliverables

- A working terminal-based library management system covering add, view, issue, return, and search operations
- Verified status transitions between Available and Issued for individual books
- A tabular catalogue view listing all books with their current status and borrower
- A working search feature returning matches across ID, title, and author fields
- Documented groundwork for future enhancements, such as due dates, fines, or persistent (file-based) storage

5.6 Core Logic

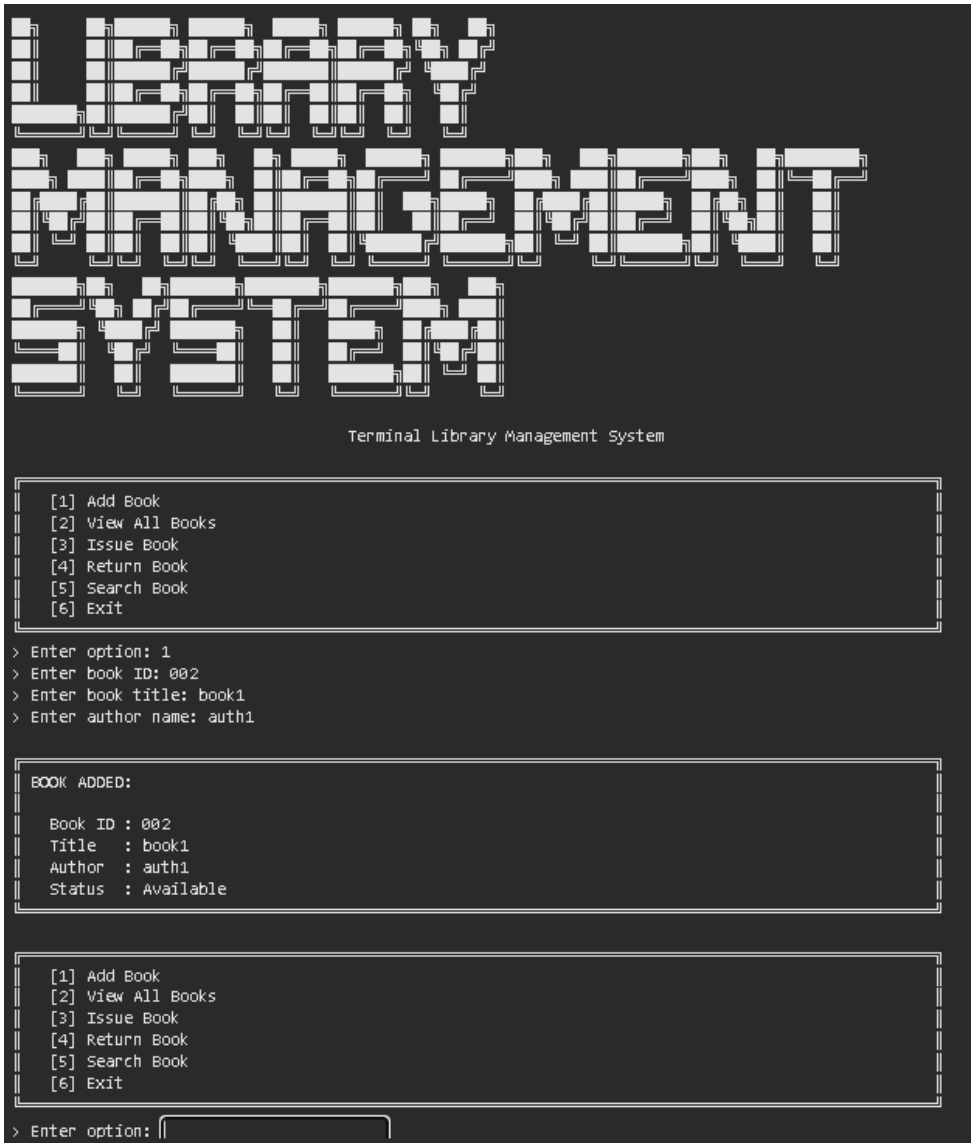
The issue_book() and return_book() functions form the core of the system, governing the state transitions of each book record.

```
def issue_book(book_id, member):
    # Only an existing, currently-Available book can be issued
    if book_id in BOOKS and BOOKS[book_id]["status"] == "Available":
        BOOKS[book_id]["status"] = "Issued"
        BOOKS[book_id]["issued_to"] = member
        return True
    return False

def return_book(book_id):
    # Only a book currently marked Issued can be returned
    if book_id in BOOKS and BOOKS[book_id]["status"] == "Issued":
        BOOKS[book_id]["status"] = "Available"
        BOOKS[book_id]["issued_to"] = ""
        return True
    return False
```

In short: a book can only move from Available to Issued, and only back from Issued to Available — each function checks the book's current status before making the change, which prevents a book from being issued twice or returned without first being issued.

5.7 Output Screenshot



6. Project Timeline

The table below traces the full laboratory progression to date. Week 3 (Programs 1–3: Bank Management System, Hotel Booking System, and Library Management System) is the subject of this combined report and is marked as the Current Week; earlier weeks are marked Completed and later weeks are marked Upcoming.

Week	Date	Activity	Status
Week 1	10/07/2026	Student Grade Management System	Completed
Week 2	17/07/2026	Program 1: Electricity Bill Calculator	Completed
Week 2	17/07/2026	Program 2: ATM Transaction Simulator	Completed
Week 3	**24/07/2026**	**Program 1: Bank Management System**	**Current Week**
Week 3	**24/07/2026**	**Program 2: Hotel Booking System**	**Current Week**
Week 3	**24/07/2026**	**Program 3: Library Management System**	**Current Week**
Week 4	31/07/2026	Program 1: Password Strength Checker	Upcoming
Week 4	31/07/2026	Program 2: Email Validation System	Upcoming
Week 5	TBD	Text Encryption & Decryption Tool	Upcoming

7. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Primary programming language used to implement all three console applications
Built-in dict & list	In-memory storage of records for accounts, bookings and library books (USERS, ACCOUNTS, BOOKINGS, ROOM_TYPES, BOOKS)
random module	Generation of random numeric components for unique bank account numbers
math module	Floor computation used while constructing account numbers
f-strings	Formatted, tabular console output across all three programs
Terminal / Command-Line Interface	Execution environment for running and testing each menu-driven program
Text Editor / IDE (VS Code / PyCharm)	Used to write and debug the Python source files

8. References

- Python Official Documentation: https://docs.python.org/3/

- Python Dictionaries Guide: https://docs.python.org/3/tutorial/datastructures.html#dictionaries
- Python random module documentation: https://docs.python.org/3/library/random.html
- Python math module documentation: https://docs.python.org/3/library/math.html
- Python f-strings & String Formatting: https://docs.python.org/3/tutorial/inputoutput.html
- Real Python — Dictionaries in Python: https://realpython.com/python-dicts/
- GeeksforGeeks — Bank / Hotel / Library Management System (Python) tutorials
- Brainware University — Machine Learning Lab Course Guidelines

9. Signatures

Signature of Student(s): **Date of Submission:**

Signature of Guide/Supervisor: